

Module 2: Configuring MCP Servers in Claude Desktop and Claude Code

Contents

- 1. Introduction — configuration files at a glance
- 2. Part A — Claude Desktop
 - 2.1 Three ways to add a server
 - 2.2 Prerequisites
 - 2.3 Locating claude_desktop_config.json
 - 2.4 Structure of the file
 - 2.5 Hands-on
 - 2.6 Custom Connectors (remote servers)
- 3. Part B — Claude Code
 - 3.1 Prerequisites
 - 3.2 Scopes
 - 3.3 Configuration files in detail
 - 3.4 Adding a stdio server
 - 3.5 Adding an HTTP server
 - 3.6 add-json
 - 3.7 Import from Claude Desktop
 - 3.8 Managing servers
 - 3.9 Environment variables
 - 3.10 Hands-on

1. Introduction

In Module 1 you watched the MCP protocol on the wire with the MCP Inspector. In this module you become the operator: you will connect real MCP servers to the two Claude hosts you use every day — **Claude Desktop** and **Claude Code**.

Both hosts launch local servers over **stdio** and reach remote servers over **Streamable HTTP**, but they keep their configuration in different files and offer different ways to add servers. This guide covers Claude Desktop completely first, and then Claude Code.

1.1 Configuration files at a glance

Keep this table handy — every file mentioned in this guide is listed here.

Host	File / place	macOS path	Windows path
Claude Desktop	claude_desktop_config.json (local servers)	~/Library/Application Support/Claude/claude_desktop_config.json	%APPDATA%\Claude\claude_desktop_config.json (Microsoft Store install: see note)
Claude Desktop	Desktop Extensions (.mcpb)	Managed by the app: Settings → Extensions	Managed by the app: Settings → Extensions

Host	File / place	macOS path	Windows path
Claude Desktop	Custom Connectors (remote)	Stored in your Claude account — no local file	Stored in your Claude account
Claude Desktop	MCP log files	~/Library/Logs/Claude/	%APPDATA%\Claude\logs\
Claude Code	~/.claude.json (user and local scope)	/Users/<you>/.claude.json	C:\Users\<you>\.claude.
Claude Code	.mcp.json (project scope)	<project root>/.mcp.json	<project root>\.mcp.json
Claude Code	settings.json (approval of project servers)	~/.claude/settings.json <project>/.claude/settings.json <project>/.claude/settings.local.json	%USERPROFILE%\.claude\s <project>\.claude\setti <project>\.claude\setti
Claude Code	managed-mcp.json (organisation, admin only)	/Library/Application Support/ClaudeCode/managed-mcp.json	C:\Program Files\Claude mcp.json

Important: Claude Desktop and Claude Code do **not** share configuration. A server added to `claude_desktop_config.json` is not visible in Claude Code, and the other way round. (Claude Code can import Desktop servers once — see section 3.7.)

2. Part A — Claude Desktop

Claude Desktop is available for **macOS** and **Windows**. (On Linux, use Claude Code — Part B.) It offers three ways to add an MCP server.

2.1 Three ways to add a server — and when to use which

Option	What it is	Where you manage it	Use it when
Desktop Extensions (.mcpb files)	A packaged local MCP server you install with one click. Claude Desktop ships its own Node.js runtime, and API keys you enter are stored in macOS Keychain / Windows Credential Manager.	Settings → Extensions	The server is published as an extension (e.g. in the extension directory), or you are a non-developer who should not edit JSON.
claude_desktop_config.json	A JSON file in which you list local servers yourself: the command to run, its arguments and environment variables.	Settings → Developer → Edit Config	You are developing or testing your own server, or using a server that is only available as an npm / PyPI package or a script on your machine.
Custom Connectors	A remote MCP server reached by URL over Streamable HTTP, usually signed in with OAuth.	Settings → Connectors → Add custom connector	The server is hosted on the internet (a SaaS product or your company's remote server) and you want it on claude.ai web and desktop alike .

Rule of thumb: local server you are building or testing

- `claude_desktop_config.json`. Ready-made local server → Desktop Extension.
- Server on the internet → Custom Connector.

2.2 Prerequisites

- Latest **Claude Desktop** installed (macOS: Claude menu → Check for Updates...).
- **Node.js LTS** for servers started with npx. Check with `node --version`.
- **uv** for Python servers started with uvx or `uv run`. Check with `uv --version`.
- A test folder (the "sandbox") the filesystem server is allowed to access.

2.3 Locating `claude_desktop_config.json`

Open it from the app (recommended)

1. **macOS:** click the **Claude** menu in the top menu bar (not the settings inside the chat window) → **Settings...**
2. **Windows:** click the menu icon (☰) at the top-left of the Claude window → **File** → **Settings...**
3. Open the **Developer** tab and click **Edit Config**. Claude creates the file if it does not exist and shows it in Finder / File Explorer. Open it in any text editor.

File locations

Operating system	Path to <code>claude_desktop_config.json</code>
macOS	<code>~/Library/Application Support/Claude/claude_desktop_config.json</code> (expands to <code>/Users/<you>/Library/Application Support/Claude/...</code> ; the Library folder is hidden — in Finder Go → Go to Folder...)
Windows — standard installer (.exe from claude.ai)	<code>%APPDATA%\Claude\claude_desktop_config.json</code> (expands to <code>C:\Users\<you>\AppData\Roaming\Claude\claude_desktop_config.json</code>)
Windows — Microsoft Store / MSIX package	<code>%LOCALAPPDATA%\Packages\Claude_pzs8srxrjxfjjc\LocalCache\Roaming\Claude\claude_desktop_config.json</code>

2.4 Structure of `claude_desktop_config.json`

All servers live under one top-level key, `mcpServers` (case-sensitive). Each entry is keyed by a name you choose; that name is what you see in Claude Desktop.

Field	Required	Meaning
<code>command</code>	Yes	The program to launch, e.g. <code>npx</code> , <code>uvx</code> , <code>uv</code> , <code>node</code> , <code>python</code> , or a full path to an executable.
<code>args</code>	No	List of command-line arguments passed to the program.
<code>env</code>	No	Environment variables for the server process (API keys, settings).

Claude Desktop launches every entry as a **subprocess** and talks to it over **stdio**.

The file is for local servers; remote servers are added as Custom Connectors (section 2.8).

Example — macOS

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/you/mcp-lab-sandbox"
      ]
    },
    "fetch": {
      "command": "uvx",
      "args": ["mcp-server-fetch"]
    }
  }
}
```

Example — Windows

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "C:\\Users\\you\\mcp-lab-sandbox"
      ]
    },
    "fetch": {
      "command": "uvx",
      "args": ["mcp-server-fetch"]
    }
  }
}
```

Path rules (the most common cause of failures)

Rule	macOS	Windows
Use absolute paths	/Users/you/mcp-lab-sandbox (not ~/mcp-lab-sandbox, not ./sandbox)	C:\\Users\\you\\mcp-lab-sandbox inside JSON (not %USERPROFILE%, not relative paths)
Escape backslashes	Not needed — paths use /	Every \ must be written as \\ in JSON. Forward slashes (C:/Users/you/...) also work.
Spaces in paths	Fine inside the JSON string	Fine inside the JSON string
Programs not found	If npx/uvx is not found, use the full path from which npx	If npx/uvx is not found, use the full path from where npx (e.g. C:\\Program Files\\nodejs\\npx.cmd)

Passing environment variables and API keys

Example — your own Python server with an API key

```
{
  "mcpServers": {
    "weather": {
      "command": "uv",
      "args": ["--directory", "/Users/you/projects/weather", "run", "server.py"],
      "env": {
        "WEATHER_API_KEY": "your-key-here"
      }
    }
  }
}
```

Security: Keys in `claude_desktop_config.json` are stored as plain text. For shared machines prefer a Desktop Extension, which stores secrets in macOS Keychain or Windows Credential Manager.

2.5 Hands-on: add the filesystem and fetch servers

Step 1 — Create the sandbox folder

macOS (Terminal)

```
mkdir -p "$HOME/mcp-lab-sandbox"
echo "Hello from MCP" > "$HOME/mcp-lab-sandbox/hello.txt"
```

Windows (PowerShell)

```
New-Item -ItemType Directory -Force -Path "$HOME\mcp-lab-sandbox"
Set-Content -Path "$HOME\mcp-lab-sandbox\hello.txt" -Value "Hello from MCP"
```

Step 2 — Check that the servers run on their own

Run the same command Claude Desktop will run. If it fails here, it will fail inside Claude Desktop too. Press Ctrl+C to stop.

```
npx -y @modelcontextprotocol/server-filesystem "$HOME/mcp-lab-sandbox"
uvx mcp-server-fetch
```

Step 3 — Edit the configuration file

1. Open **Settings → Developer → Edit Config** (section 2.3).
2. Replace the contents with the macOS or Windows example from section 2.4, using your real user name in the path.
3. Save the file. Make sure it is valid JSON (no trailing commas, matching braces).

Step 4 — Fully restart Claude Desktop

The configuration is read **only when the app starts**. Closing the window is not enough.

macOS	Windows
Claude menu → Quit Claude (or Cmd+Q), then open Claude again.	Right-click the Claude icon in the system tray (bottom-right, near the clock) → Quit , then open Claude again. Closing the window leaves Claude running in the tray.

Step 5 — Verify that the servers loaded

- In the chat box, click the + button (bottom-left) → **Connectors → Manage connectors**. filesystem and fetch should be listed; click one to see its tools.
- Or open **Settings → Developer**: each local server is shown with its status (running or failed).

Step 6 — Use the servers

- Ask: "Read hello.txt from my mcp-lab-sandbox folder."
- Ask: "Fetch modelcontextprotocol.io/llms.txt and summarise it."

Before each tool runs, Claude Desktop shows an **approval prompt** with the exact tool and arguments. Choose **Allow once**, **Always allow** for that tool, or **Deny**. Review every request — the server runs with your user account's permissions.

2.6 Remote servers — Custom Connectors

A remote MCP server runs on the internet and is reached by URL over Streamable HTTP. In Claude Desktop you add it as a **Custom Connector**, not in `claude_desktop_config.json`.

Pro and Max plans

1. Open **Settings** → **Connectors** (also reachable from + → **Connectors** → **Manage connectors** in the chat box).
2. Click **Add custom connector**.
3. Enter a name and the server URL, e.g. `https://mcp.example.com/mcp`.
4. If the server needs a pre-registered OAuth app, open **Advanced settings** and enter the **OAuth Client ID** and **Client Secret**.
5. Click **Add**, then **Connect** and complete the sign-in in your browser.

Team and Enterprise plans

Only an **Owner** can add a custom connector, in **Admin settings** → **Connectors**. Members then find it in their Connectors list (marked Custom) and click **Connect** to sign in with their own account.

Things to know

- Connectors are saved in your **Claude account**, not on your computer, so they are the same on macOS, Windows and claude.ai on the web.
- The connection is made from Anthropic's cloud, so the URL must be reachable from the internet. A server on localhost cannot be a Custom Connector — run it locally through `claude_desktop_config.json` instead.
- Only connect to servers you trust and review the permissions requested during sign-in.

3. Part B — Claude Code

Claude Code is Anthropic's agentic coding tool for the terminal (also used inside VS Code, JetBrains and the desktop app's Code tab). It runs on **macOS, Linux and Windows**. You manage its MCP servers with the `claude mcp` command or by editing JSON files.

3.1 Prerequisites

- Claude Code installed. macOS / Linux: `curl -fsSL https://claude.ai/install.sh | bash`. Windows (PowerShell): `irm https://claude.ai/install.ps1 | iex`.
- Check with `claude --version`, then run `claude` once to sign in.
- Node.js and/or uv, as for Claude Desktop, for servers started with `npx` / `uvx`.

3.2 Scopes — where a server is saved

Every server you add belongs to a **scope**. The scope decides which file stores it, which projects it loads in, and whether your team gets it too.

Scope	Loads in	Shared with team?	Stored in	Use it when
local (default)	Only the current project	No	<code>~/.claude.json</code> , under <code>projects</code> → " <code><project path></code> " → <code>mcpServers</code>	Personal or experimental servers, or servers with your own credentials, for one project.

Scope	Loads in	Shared with team?	Stored in	Use it when
project	Only the current project	Yes — committed to git	.mcp.json in the project root	Servers everyone working on the repository should have (e.g. the project's database or issue tracker).
user	All your projects	No	~/.claude.json, top-level mcpServers	Personal tools you want everywhere (e.g. fetch, a docs server).
managed	All projects for every user on the machine	Set by IT	managed-mcp.json (system folder)	Organisations that must control which servers are allowed. Not edited by students.

Which one wins if the same name is used twice?

If a server with the same name exists in several places, Claude Code uses the one with the highest priority and ignores the others completely (fields are not merged):

Highest	1. local scope (~/ .claude.json → projects → <path>)
	2. project scope (.mcp.json)
	3. user scope (~/ .claude.json → top-level mcpServers)
	4. servers provided by plugins
	5. claude.ai connectors
Lowest	(servers your organisation provides through managed settings rank above all of these)

3.3 The Claude Code configuration files in detail

~/ .claude.json — user and local scopes

macOS / Linux	Windows
~/ .claude.json (/Users/<you>/ .claude.json)	%USERPROFILE%\ .claude.json (C:\Users\<you>\ .claude.json)

This is a **single file**, not a folder. Claude Code also keeps your sign-in and other state in it, so prefer the `claude mcp` commands over editing it by hand. User-scope servers sit in the top-level `mcpServers`; local-scope servers sit under the project's path:

Shape of ~/ .claude.json (MCP parts only)

<pre>{ "mcpServers": { "fetch": { "type": "stdio", "command": "uvx", "args": ["mcp-server-fetch"] } }, "projects": { "/Users/you/code/shop-app": { "mcpServers": { "stripe": { "type": "http", "url": "https://mcp.stripe.com" } } } } }</pre>
--

On Windows the project key is the Windows path, for example "C:\\Users\\you\\code\\shop-app".

.mcp.json — project scope

Lives in the root of the project and is meant to be committed to git. It uses the same mcpServers shape as Claude Desktop, and can hold stdio and HTTP servers side by side:

Example .mcp.json

```
{
  "mcpServers": {
    "filesystem": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "${CLAUDE_PROJECT_DIR}"]
    },
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/",
      "headers": { "Authorization": "Bearer ${GITHUB_PAT}" }
    }
  }
}
```

- **Environment variables:** \${VAR} and \${VAR: -default} are expanded in command, args, env, url and headers, so secrets stay out of git. \${CLAUDE_PROJECT_DIR} is the project root.
- **Approval:** the first time you open a project with a .mcp.json, Claude Code asks whether to trust its servers. To reset your choices run `claude mcp reset-project-choices`.
- **Same file on both operating systems:** because the file is shared, avoid hard-coded paths such as /Users/... or C:\...; use \${CLAUDE_PROJECT_DIR} or variables instead.

settings.json — approving and blocking project servers

File	Directory	Who it affects
User settings	~/.claude/settings.json	You, in every project
Shared project settings	<project>/.claude/settings.json	Everyone (committed to git)
Personal project settings	<project>/.claude/settings.local.json	You, in this project (not committed via .gitignore)

MCP-related keys you can put in these files:

- `enableAllProjectMcpServers`: `true` — trust every server in the project's .mcp.json without asking.
- `enabledMcpjsonServers`: `["github"]` — trust only the listed .mcp.json servers.
- `disabledMcpjsonServers`: `["filesystem"]` — never load the listed .mcp.json servers.

Example .claude/settings.local.json

```
{
  "enabledMcpjsonServers": ["github"],
  "disabledMcpjsonServers": ["filesystem"]
}
```

managed-mcp.json — organisation-controlled servers (for awareness)

macOS	Linux / WSL	Windows
/Library/Application Support/ClaudeCode/managed-mcp.json	/etc/claude-code/managed-mcp.json	C:\Program Files\ClaudeCode\managed-mcp.json

Deployed by IT with administrator rights (MDM, Group Policy, Intune). When this file exists, users get exactly the servers it lists and `claude mcp add` is refused. An empty `{"mcpServers": {}}` turns MCP off. Organisations that only want to filter servers use `allowedMcpServers` / `deniedMcpServers` in managed settings instead.

3.4 Adding a local (stdio) server with `claude mcp add`

Syntax

```
claude mcp add [options] <name> -- <command> [args...]
```

Options (must come BEFORE <name>):

<code>--scope local project user</code>	default: local
<code>--transport stdio</code>	default for a command; can be omitted
<code>--env KEY=value</code>	repeat for several variables

The `--` separates Claude Code's own options from the command that starts the server. Everything after `--` is passed to the server unchanged.

macOS / Linux examples

```
# filesystem server, available in all your projects
claude mcp add --scope user filesystem -- npx -y @modelcontextprotocol/server-filesystem
"$HOME/mcp-lab-sandbox"

# fetch server (Python, via uvx)
claude mcp add --scope user fetch -- uvx mcp-server-fetch

# your own Python server with an API key, this project only
claude mcp add --env WEATHER_API_KEY=abc123 weather -- uv --directory
/Users/you/projects/weather run server.py
```

Windows (PowerShell) examples

```
# Use forward slashes in the folder path
$sandbox = "$HOME\mcp-lab-sandbox" -replace '\\', '/'
claude mcp add --scope user filesystem -- npx -y @modelcontextprotocol/server-filesystem
"$sandbox"

claude mcp add --scope user fetch -- uvx mcp-server-fetch
```

Windows tip: If a server started with `npx` fails to connect on native Windows ("Connection closed"), wrap the command with `cmd /c`: `claude mcp add fetch-js -- cmd /c npx -y <package>`.

3.5 Adding a remote (HTTP) server

```
# Remote server that signs in with OAuth
claude mcp add --transport http sentry https://mcp.sentry.dev/mcp

# Remote server with a token in a header, shared with the team
claude mcp add --transport http --scope project github https://api.githubcopilot.com/mcp/ --
header "Authorization: Bearer YOUR_PAT"
```

For OAuth servers, start Claude Code, type `/mcp`, select the server and choose **Authenticate**; a browser window opens to sign in. You can also run `claude mcp login <name>` from the terminal.

If you use `claude.ai` Custom Connectors (section 2.8), Claude Code can use those too once you are signed in with the same account; they appear in `/mcp`.

3.6 Adding a server from JSON

`claude mcp add-json` takes the exact JSON entry. It is handy when copying a definition from documentation or from `claude_desktop_config.json`.

macOS / Linux (single quotes)

```
claude mcp add-json local-fetch '{"type":"stdio","command":"uvx","args":["mcp-server-fetch"]}'

claude mcp add-json weather
'{"type":"http","url":"https://api.example.com/mcp","headers":{"Authorization":"Bearer t"}}'
```

Windows PowerShell (escape the inner double quotes)

```
claude mcp add-json local-fetch '{"\"type\": \"stdio\", \"command\": \"uvx\", \"args\": [\"mcp-server-fetch\"]}'
```

PowerShell 7.3 or later passes the quotes correctly, so the macOS form works unchanged. Windows PowerShell 5.1 needs the inner quotes escaped as shown.

3.7 Importing servers from Claude Desktop

```
claude mcp add-from-claude-desktop
```

Shows the servers in `claude_desktop_config.json` and lets you choose which to copy. Available on **macOS** and **WSL** only.

3.8 Managing servers

Command	Where	What it does
<code>claude mcp list</code>	Terminal	Lists every configured server with its connection status.
<code>claude mcp get <name></code>	Terminal	Shows one server's scope, command or URL.
<code>claude mcp remove <name></code>	Terminal	Removes the server (add <code>--scope</code> if the name exists in several scopes).
<code>claude mcp reset-project-choices</code>	Terminal	Forgets which <code>.mcp.json</code> servers you approved or rejected.
<code>/mcp</code>	Inside a Claude Code session	Status and tool counts, authenticate OAuth servers, reconnect, or enable/disable a server for this project. It does not add servers.

Unlike Claude Desktop, you do not restart an application: start a new `claude` session (or use `/mcp` to reconnect) and the changes are picked up.

3.9 Useful environment variables

Variable	Purpose
<code>MCP_TIMEOUT</code>	How long (ms) to wait for servers to start, e.g. <code>MCP_TIMEOUT=10000 claude</code> .
<code>MAX_MCP_OUTPUT_TOKENS</code>	Maximum size of a tool result (default 25,000 tokens).

On Windows PowerShell set them first: `$env:MCP_TIMEOUT = "10000"; claude`.

3.10 Hands-on: filesystem and fetch in Claude Code

1. Create the sandbox folder as in section 2.5, Step 1 (skip if you already have it).
2. Add both servers at **user** scope with the commands in section 3.4 for your operating system.
3. Run `claude mcp list` — both servers should show as connected. Run `claude mcp get filesystem` to see where it was saved.
4. Open `~/.claude.json` (macOS) or `%USERPROFILE%\claude.json` (Windows) and find the two entries under the top-level `mcpServers`.
5. Create a project folder and add a project-scope server: `mkdir mcp-m2-project`, `cd mcp-m2-project`, `claude mcp add --scope project docs -- uvx mcp-server-fetch`. Open the new `.mcp.json` — this is the file you would commit for your team.
6. Start `claude` in the project folder, approve the project server when asked, then type `/mcp` to see all three servers.
7. Ask Claude: "Read `hello.txt` from my `mcp-lab-sandbox`" and "Fetch `modelcontextprotocol.io/llms.txt` and summarise it". Approve each tool call.
8. Optional clean-up: `claude mcp remove filesystem`, `claude mcp remove fetch`, `claude mcp remove docs --scope project`.

4. Quick decision guide

I want to...	Host	Use
Try a ready-made local server without editing JSON	Claude Desktop	Desktop Extension (Settings → Extensions)
Test my own local server while I develop it	Claude Desktop	<code>claude_desktop_config.json</code>
Connect a hosted (internet) MCP server	Claude Desktop	Custom Connector (Settings → Connectors)
Have a server in every project, just for me	Claude Code	<code>claude mcp add --scope user</code> → <code>~/.claude.json</code>
Have a server in one project, just for me (own credentials)	Claude Code	<code>claude mcp add (local scope)</code> → <code>~/.claude.json</code> under the project
Share a server with everyone on the repository	Claude Code	<code>claude mcp add --scope project</code> → <code>.mcp.json</code> (commit it)
Control which servers everyone in the company may use	Claude Code (IT)	<code>managed-mcp.json</code> or allow/deny lists in managed settings
Re-use my Claude Desktop servers in Claude Code	Claude Code	<code>claude mcp add-from-claude-desktop</code> (macOS/WSL) or <code>add-json</code>